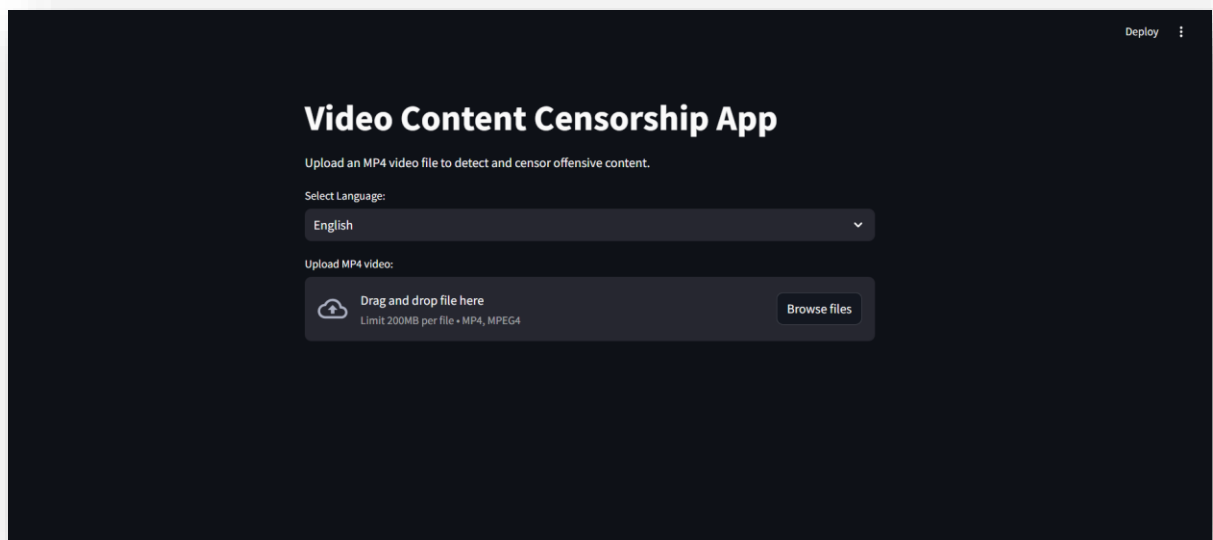


Comprehensive Report on the Video Content Censorship Application

1. Executive Summary

The **Video Content Censorship Application** is designed to automatically detect and censor offensive content in video files. It integrates advanced natural language processing (NLP) and speech-to-text technologies to identify and classify offensive language, then modifies the video's audio by replacing toxic words with a beep sound. This tool supports both **English** and **Spanish** languages and uses a combination of predefined lists of offensive words and AI-based classification models to ensure thorough detection. Once processed, the application enables users to download the censored video directly.



2. Key Features and Capabilities

- **Video Upload:** Simple drag-and-drop file uploader to add MP4 video files for processing.
- **Multi-language Support:** Supports both **English** and **Spanish** for offensive content detection.

Video Content Censorship App

Upload an MP4 video file to detect and censor offensive content.

Select Language:

English

English

- **Real-time Transcription:** Uses OpenAI's **Whisper** model to transcribe the video's audio and extract word-level timestamps.

English

Upload MP4 video:



Drag and drop file here

Limit 200MB per file • MP4, MPEG4

Browse files



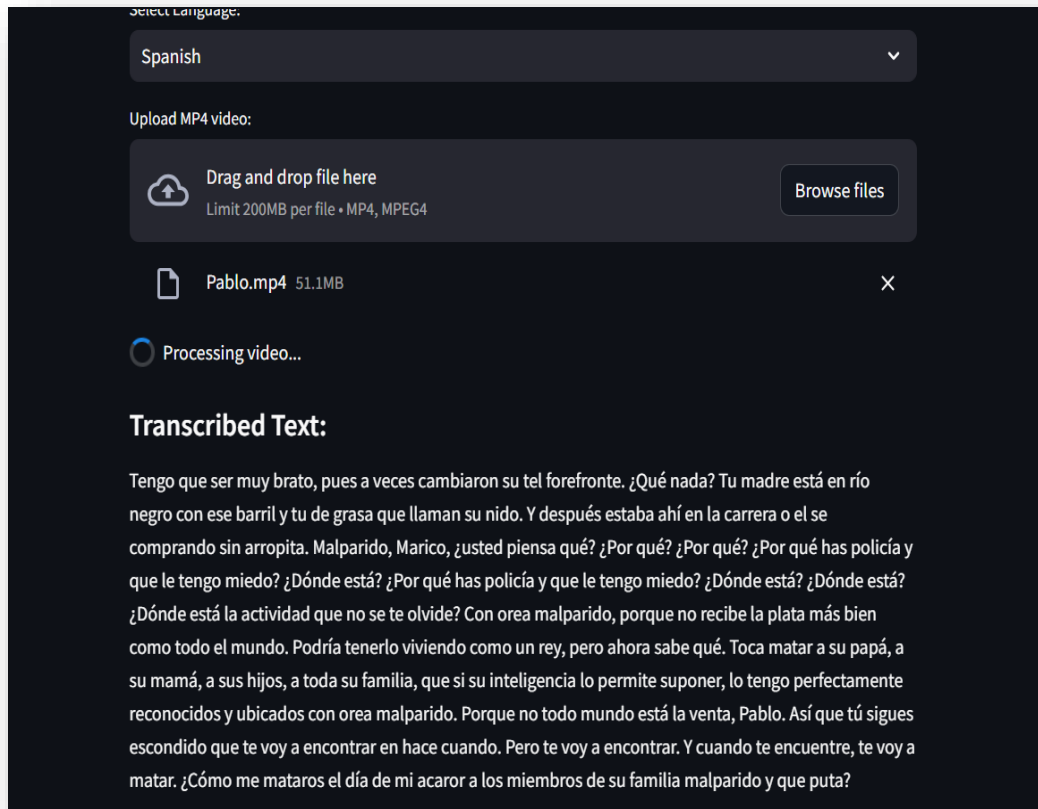
GOT.mp4 12.3MB



Processing video...

Transcribed Text:

Father, I wish to confess. I wish to confess. You wish to confess? I saved you. I saved the city and all your worthless lives. I should have let Stannis kill you all. I saved you. Do you wish to confess? Yes, Father. I'm guilty. Guilty? Is that what you want to hear? You admit you poisoned the king. No. Of that I'm innocent. I'm guilty of a far more monstrous crime. I'm guilty of being a dwarf. You're not on trial for being a dwarf. Yes, I am. I've been on trial for that my entire life. Have you nothing to say in your defence? Nothing but this. I did not do it. I did not kill Joffrey, but I wish that I had. Watching your vicious bastard die gave me more relief than a thousand lying holes. I wish I was the monster you think I am. I wish I had enough poison for the whole pack of you. I would gladly give my life to what you all swallow it. The Marin! The Marin! That's got to crisp her back to his soul. I will not give my life for Joffrey's murder. And I know I'll get no justice here. So I will let the gods decide my fate. I demand a trial by combat. I demand a trial by combat.



- **Offensive Language Detection:**
- **English:**

Detected Offensive Words:

smoked, pheromones, hookah, fucking, assholes, goddammit, goddammit, blow, piss, die, death, murder, dumb, kill, kill, fucking, motherfucker, fucking, kill, dammit

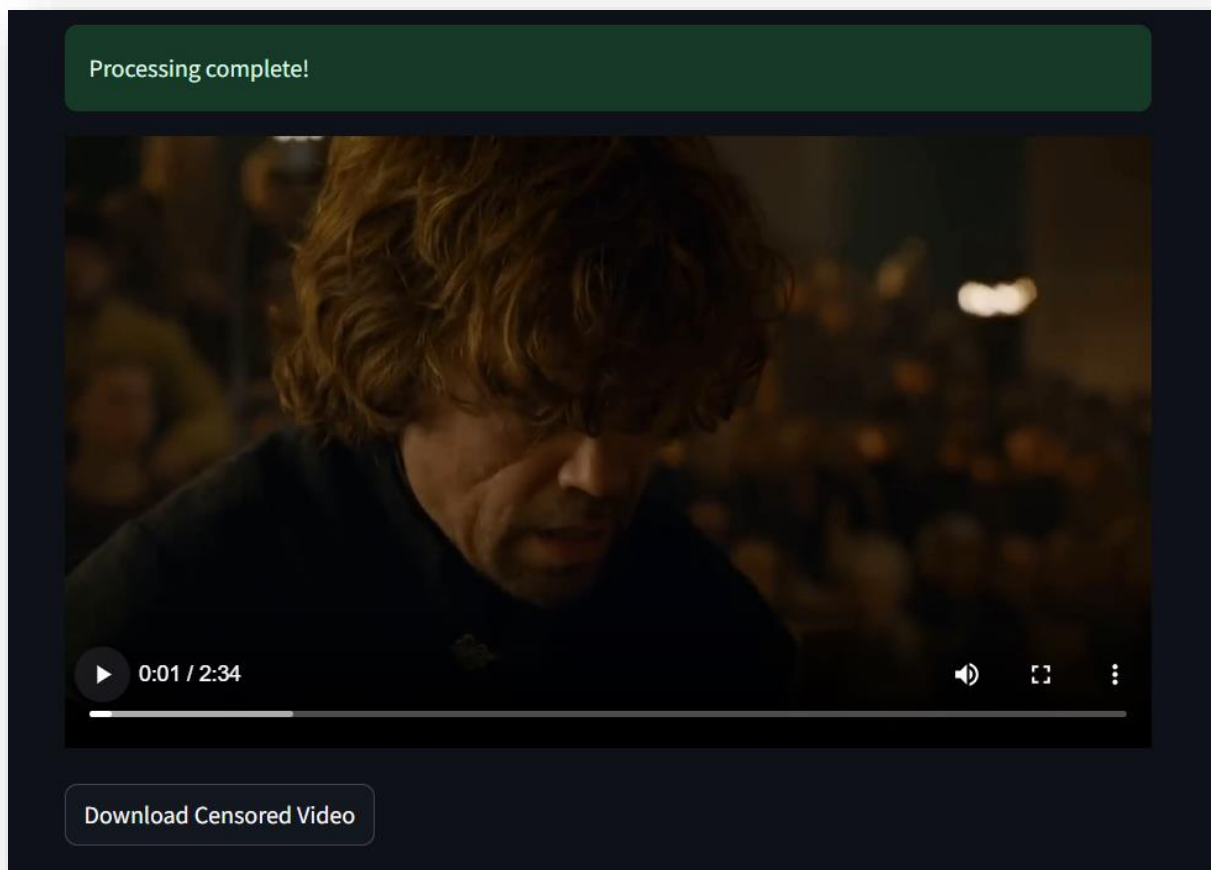
Spanish:

Detected Offensive Words:

matar, matar

- For **English**, it combines a **predefined list** of offensive words with a **machine learning model** (unitary/toxic-bert) for toxicity classification.
- For **Spanish**, the app uses a **predefined list** of offensive words and the **Spanish-specific model** (jorgeortizfuentes/spanish-offensive-language-bert-base-spanish-wwm-cased).

- **Content Censorship:** Detected offensive words are automatically replaced with a "beep" sound. The beep's duration is adjusted to match the length of the offensive word.
- **Final Video Creation:** The tool merges the original video with the censored audio and provides the user with a downloadable MP4 file.



- **User-Friendly Interface:** Built using **Streamlit**, the application provides a simple web interface for users to upload their videos, view transcription results, and download censored versions of the video.

3. Detailed Workflow

1. **Upload Video and Select Language:**
 - Users select the language of the video (English or Spanish) and upload the MP4 file.
2. **Audio Extraction:**
 - The **moviepy** library extracts audio from the video, saving it in WAV format for easier processing.

3. Noise Reduction:

- The audio undergoes noise reduction using **pydub**, which normalizes the audio for better transcription accuracy.

4. Transcription with Whisper:

- The **Whisper** model transcribes the audio to text, providing both a complete transcription and word-level timestamps for each word spoken in the video.

5. Offensive Word Detection:

- **For English:**
 - The application checks each word against a **predefined list** of offensive English words (english_offensive_words.txt).
 - If the word is not found in the list, it is passed through the **unitary/toxic-bert**
 - **unitary/toxic-bert: Trained models & code to predict toxic comments on 3 Jigsaw challenges: Toxic comment classification, Unintended Bias in Toxic comments, Multilingual toxic comment classification.**
 - **Built by [Laura Hanu](#) at [Unitary](#), where we are working to stop harmful content online by interpreting visual content in context.**
 - model to classify the word as toxic or non-toxic.
 - The toxicity classification is based on a confidence score; if the score exceeds 0.7, the word is flagged as toxic.
- **For Spanish:**
 - The application checks the transcription against a predefined list of Spanish offensive words (spanish_offensive_words.txt).
 - Additionally, the **spanish-offensive-language-bert-base-spanish-wwm-cased** model is used to detect offensive or toxic content.
 - **spanish-offensive-language-bert-base-spanish-wwm-cased**
 - This model is a fine-tuned version of dccuchile/bert-base-spanish-wwm-cased on the jorgeortizfuentes/spanish-offensive-language dataset

6. Content Censorship:

- When an offensive word is detected, the corresponding audio segment is replaced with a beep sound.
- The beep's duration is adjusted to match the duration of the offensive word. If the beep sound is too short, it is repeated; if it's too long, it is truncated.

7. Final Video Creation:

- The processed (censored) audio is combined with the original video using **moviepy**.
- The final censored video is generated and made available for download.

8. Download Censored Video:

- Once the video is processed, the user can download the final version with censored content directly from the app interface.

4. Technical Implementation

- **Streamlit:**
 - This is used as the framework for the web-based interface. It allows for seamless integration of file uploads, text display, and video display/downloads.
- **Whisper (OpenAI):**
 - Whisper transcribes audio to text and provides word-level timestamps. This feature is crucial for linking offensive words with specific time frames in the audio.
- **Transformers (Hugging Face):**
 - unitary/toxic-bert (English model) and jorgeortizfuentes/spanish-offensive-language-bert-base-spanish-wwm-cased (Spanish model) are used for the classification of toxic words based on NLP.
- **Pydub:**
 - Used for **audio manipulation**, including normalizing the audio and adjusting beep sound durations to match offensive words.
- **MoviePy:**
 - Extracts audio from video files and allows for the combination of the final censored audio with the video file.
- **Beep Sound:**
 - A standard **beep.mp3** sound is used to replace detected offensive words in the audio.

5. Challenges and Considerations

- **Accuracy of Offensive Word Detection:**
 - The system relies heavily on both predefined lists and machine learning models to detect offensive language. While these models are robust, they may not catch all instances of subtle toxicity or regional variations in offensive language.
 - To mitigate this, it's suggested that both the word lists and models be **periodically updated** based on feedback and new data.

- **Handling Edge Cases:**
 - If the **beep sound file** is not found, the user receives an error message indicating the issue.
 - **Large videos** may take a long time to process, especially when performing multiple steps like transcription, classification, and audio processing. A **progress bar** or **background processing** can be implemented for a better user experience.
- **Real-Time Processing:**
 - Video and audio processing can be slow, especially for longer videos. To make the process faster, the application could implement **parallel processing** or optimize the transcription and classification pipelines.
- **False Positives/Negatives:**
 - The models used to classify words as toxic may occasionally produce false positives (non-toxic words flagged as toxic) or false negatives (toxic words missed). Therefore, manual review may be necessary for critical content.

6. Improvements and Future Enhancements

- **Language Expansion:**
 - Currently, the app supports only **English** and **Spanish**. Supporting more languages (e.g., French, German, etc.) would require adding corresponding offensive word lists and models.
- **Real-Time Feedback:**
 - Users could benefit from **real-time feedback** on offensive words as they upload the video or during the transcription process.
- **Interactive Review Interface:**
 - Allow users to **review** the detected offensive words and manually adjust the censorship. This could be a useful feature for cases where the model might misclassify certain words.
- **Model Fine-tuning:**
 - The models for offensive language detection could be fine-tuned on **domain-specific datasets** (e.g., social media posts, online comments, etc.) to improve accuracy in identifying context-specific offensive language.

7. Conclusion

The **Video Content Censorship Application** is an effective tool for ensuring that video content adheres to community guidelines by detecting and censoring offensive language. The combination of predefined lists and machine learning models provides robust language detection, while the beep

sound ensures that offensive content is appropriately censored. The integration of technologies such as **Whisper** for transcription, **Hugging Face** models for toxic language detection, and **MoviePy** for video processing makes this tool both versatile and user-friendly.